

Detection latency holds near 2.0 s from 3 to 9 observer nodes, once a trial-count overflow bug stopped silently timing out half the run.

Full-suite results (latency, baseline, cold-start, throughput) at 500 trials per topology, before and after fixing an IPv4 octet overflow that was discarding every trial past #255.

RUN WINDOW	DURATION	TOPOLOGIES	TRIALS / TOPOLOGY
2026-08-28 17:25 - 21:07 UTC	3h 42m (was 17h 38m)	3, 5, 7, 9 nodes	500
FIX COMMIT	CHAINCODE PATH		
7dc6efd	RecordARPEntry (legacy)		

01 THE BUG

The first 500-trial run looked bimodal: roughly half of every latency and baseline sample resolved in ~2 s, and the other half sat at the 30 s detection timeout — a **49% failure rate, identical across all four node counts**, and starting at the exact same trial index every time.

ROOT CAUSE

`benchmark.go` built each trial's synthetic ARP address by writing the raw trial number straight into the last IPv4 octet. Octets only hold 0–255, so trial 256 onward produced invalid addresses — silently undetectable, forever.

```

- ip := fmt.Sprintf("10.250.%d.%d", nodeCount, trial)
+ ip := fmt.Sprintf("10.250.%d.%d", nodeCount+trial/256, trial%256)

- ip := fmt.Sprintf("10.252.0.%d", trial)
+ ip := fmt.Sprintf("10.252.%d.%d", trial/256, trial%256)

```

Confirmed pattern (pre-fix, latency & baseline)

NODE COUNT	FIRST TIMEOUT	TOTAL TIMEOUTS	RATE
3	trial 256	245 / 500	49.0%
5	trial 256	245 / 500	49.0%
7	trial 256	245 / 500	49.0%
9	trial 256	245 / 500	49.0%

Verified in all 8 affected files (latency × baseline, × 4 node counts) — deterministic, not sampling noise. Throughput and cold-start were unaffected; both already wrapped their counters into two octets.

02 AFTER THE FIX

Same 500-trial × 4-topology run, same server, same chaincode — wrapped the octet the same way cold-start's own address generator already did. The rerun finished in **3h 42m**, less than a quarter of the buggy run's 17h 38m, because no trial ever burns 30 s waiting on an address the node could never resolve.

TIMEOUTS, POST-FIX	TIMEOUTS, PRE-FIX	RUNTIME, POST-FIX	RUNTIME, PRE-FIX
0 / 4,000 trials	1,960 / 4,000	3h 42m	17h 38m

03 DETECTION LATENCY

Time from transaction submit until every observing node's local ARP cache reflects the new binding. Mean drifts down slightly as node count grows — medians hold flat near 2.0 s through node count 7, then step down at 9.

NODES	TRIALS	MEAN	MEDIAN	P95	P99
3	500	1984.9 ms	2012.5 ms	2015.0 ms	2016.0 ms
5	500	1959.3 ms	2013.0 ms	2018.0 ms	2019.0 ms
7	500	1906.2 ms	2012.0 ms	2023.0 ms	2025.0 ms
9	500	1708.2 ms	1635.0 ms	2014.0 ms	2033.0 ms

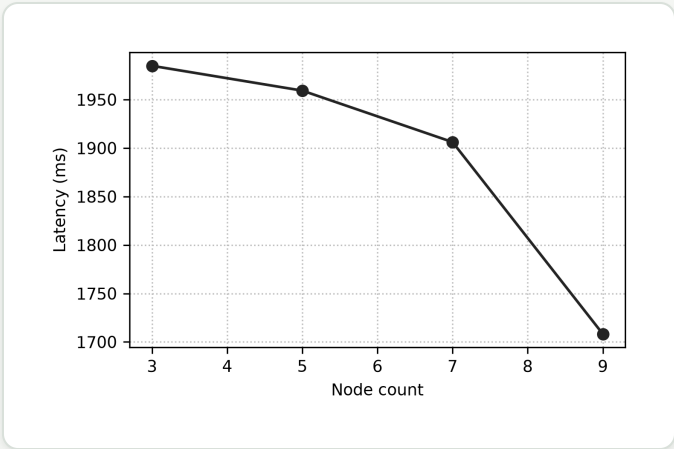


Fig. 3.1 – mean latency vs. node count

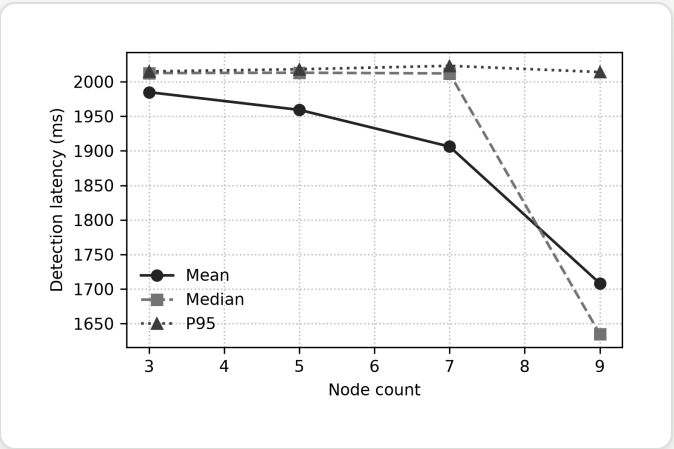


Fig. 3.2 – mean / median / P95 overlay

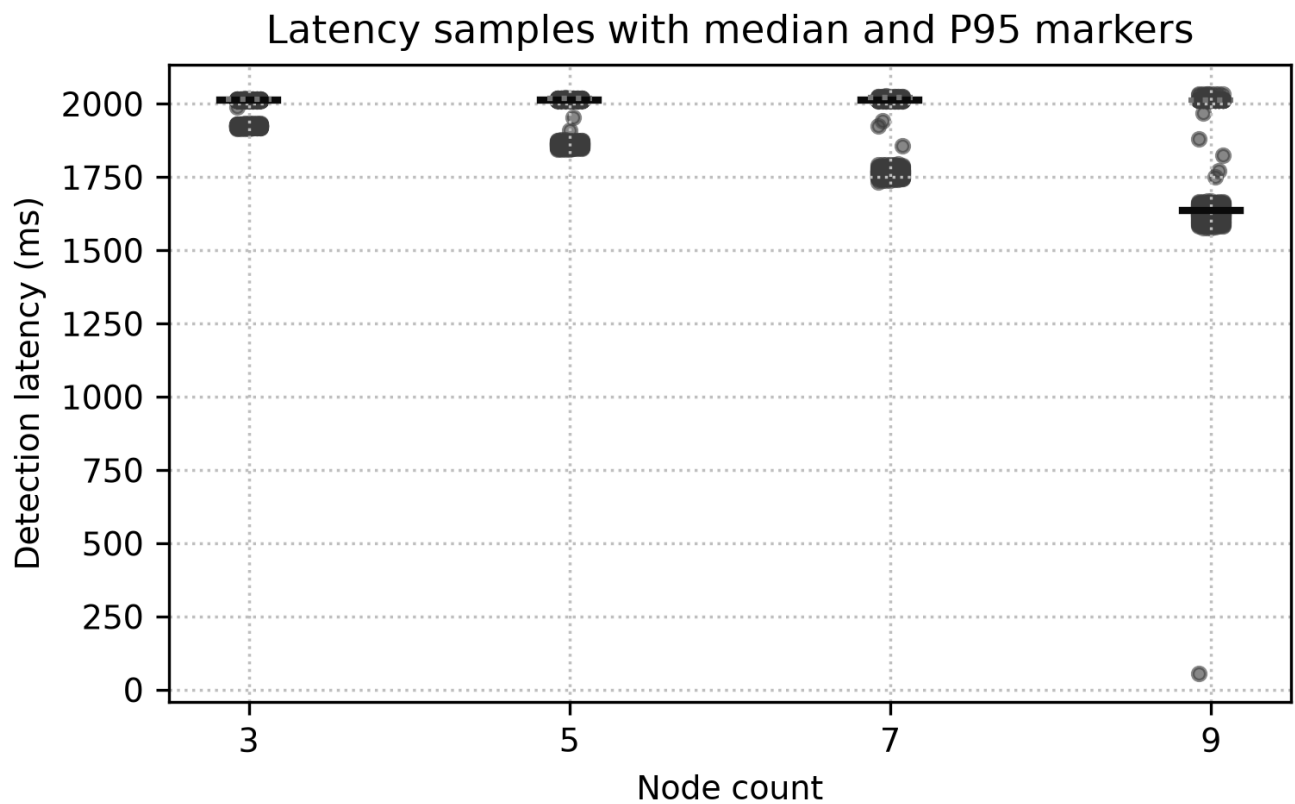


Fig. 3.3 – per-trial sample distribution, median and P95 markers per topology

04 BASELINE (PROTECTED PATH)

Same detection-latency metric measured against the protected registration path directly (no separate device-arrival framing). Tracks the latency figures closely, as expected since both exercise the same commit-then-observe pipeline.

NODES	TRIALS	MEAN	MEDIAN	P95	P99
3	500	1987.0 ms	2013.0 ms	2015.0 ms	2017.0 ms
5	500	1951.2 ms	2013.0 ms	2020.0 ms	2023.0 ms
7	500	1774.8 ms	1698.5 ms	2029.0 ms	2031.0 ms
9	500	1669.5 ms	1543.0 ms	2037.0 ms	2040.0 ms

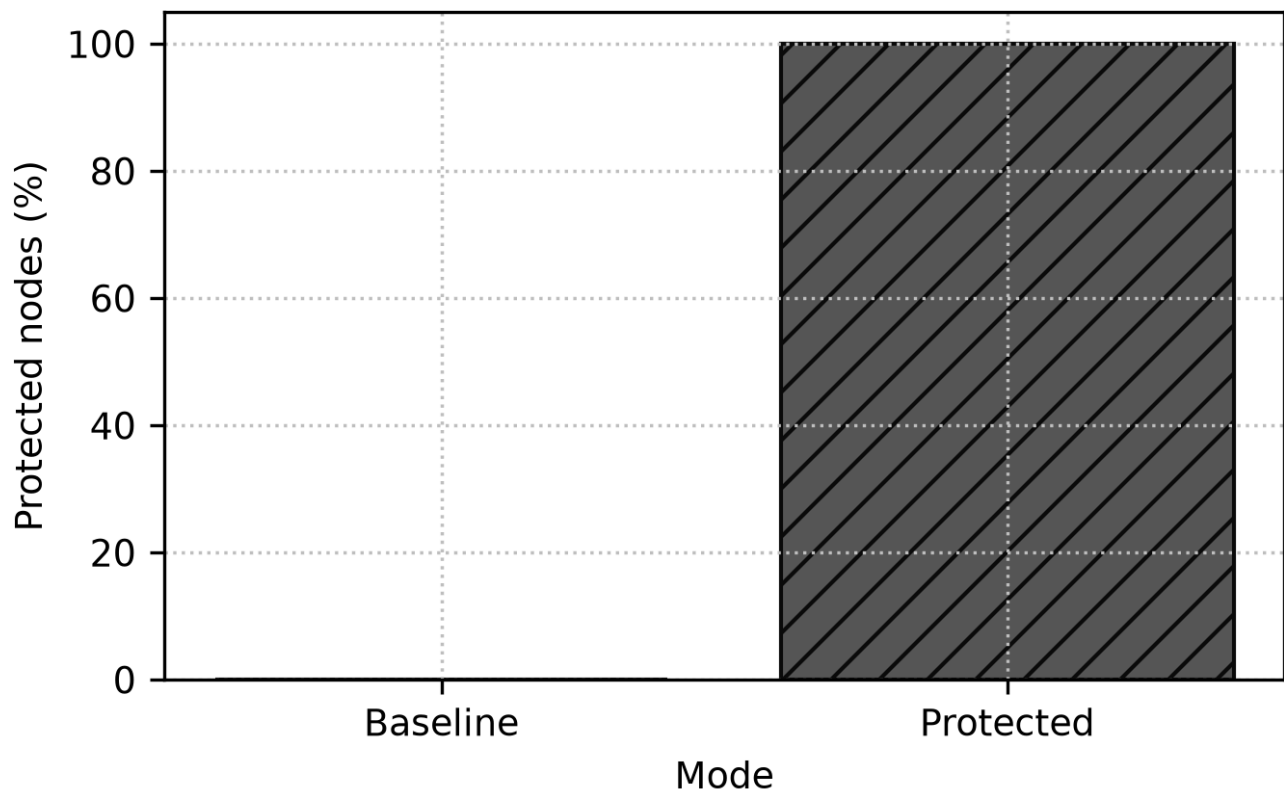


Fig. 4.1 – protected-path confirmation rate: 100% across all 500×4 trials, zero rejections

05 COLD-START SYNC

Ledger query time (`GetAllARPEntries`) plus a conservative per-entry install estimate.

NODES	SIZE 10	SIZE 50	SIZE 100	SIZE 250
3	1262 ms	1347 ms	1449 ms	1756 ms
5	3783 ms	3783 ms	3783 ms	3783 ms
7	4794 ms	4791 ms	4792 ms	4791 ms
9	5806 ms	5806 ms	5807 ms	5805 ms

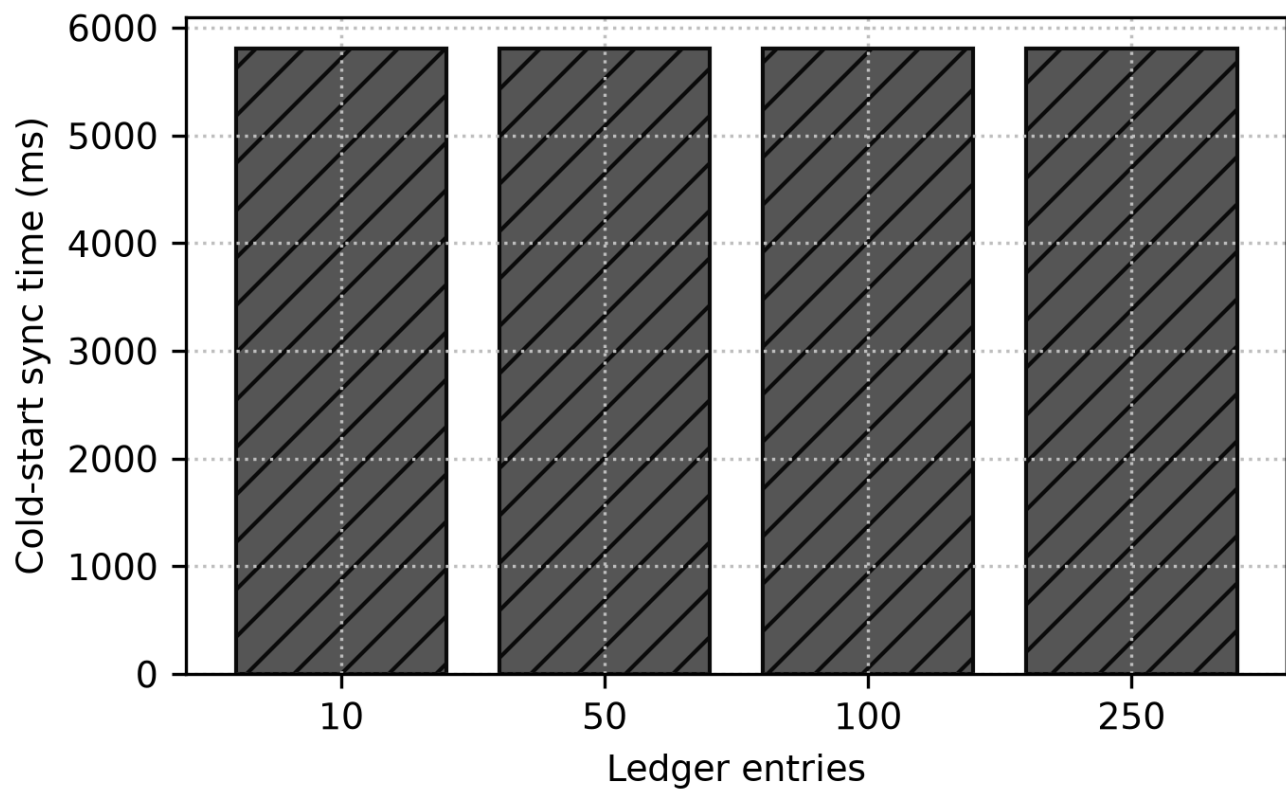


Fig. 5.1 – cold-start sync time by requested ledger size, node count 9 (most recent source file)

READ WITH CAUTION

Within any one node count, mean install time is flat across all four requested sizes — e.g. node count 5 gives ~3783 ms at size 10 *and* size 250. The driver never resets the ledger between node-count phases, so by node count 9 the ledger already carries every entry accumulated since node count 3. The cross-node-count jump (1.3–1.8 s → 3.8 s → 4.8 s → 5.8 s) is real and reproducible; the within-run "vs. requested size" comparison is confounded by that accumulation and shouldn't be read at face value.

06 THROUGHPUT UNDER LOAD

Detection latency while events are submitted at increasing rates (1–91 events/s, step 10, 10 events per rate). Latency stays essentially flat across

the sweep at every node count — no queueing blow-up observed in this rate range.

NODES	MEAN @ 1/S	MEAN @ 91/S
3	1964.1 ms	2012.5 ms
5	1932.5 ms	1914.2 ms
7	1989.5 ms	1800.5 ms
9	1897.1 ms	1772.0 ms

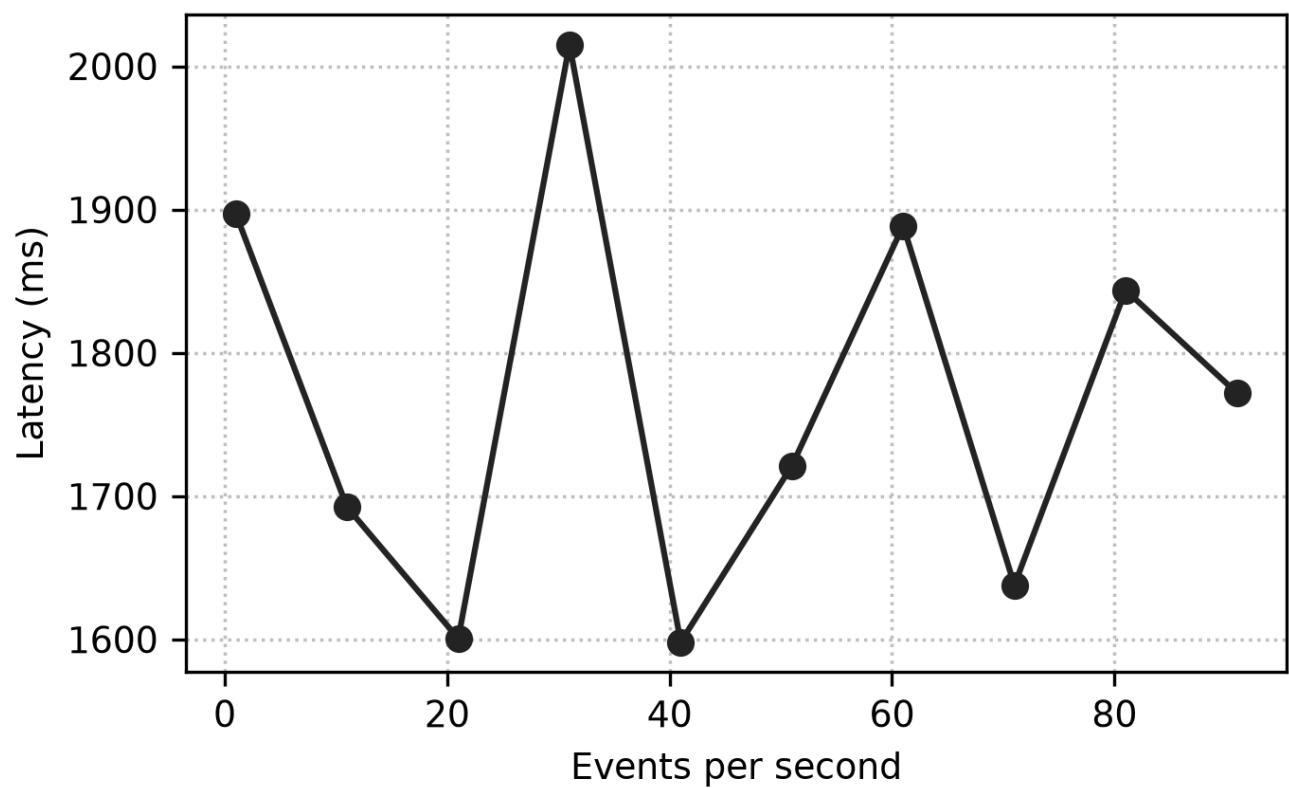


Fig. 6.1 – latency vs. submission rate, node count 9 (most recent source file)

Source data: `benchmark/remote-results-20260828-fixed/`

Run ID `20260828T172530Z` · commits `7dc6efd`, `c089aec`, `65cc5dc`

